Chapitre 8

Algorithmes probabilistes.

Exemple 1: Vérification de produit.

Instance: $(A, B, C) \in \mathscr{M}_n(\mathbb{Z}/2\mathbb{Z})^3$ **Question:** Est-ce que $AB = C$?
On propose un algorithme probabiliste : choisir un vecteur $X \in \mathscr{M}_{n,1}(\mathbb{Z}/2\mathbb{Z})$ aléatoire, et comparer ABX et CX.
On cherche à étudier $\mathbb{P}(ABX = CX \mid AB \neq C)$. L'algorithme est en $O(n^2)$.
On considère $AB \neq C$, alors $AB - C = D \neq 0$, donc il existe $d_{i,j} = 1$.
On a $ABX = CX \iff DX = 0$, et $DX = \left(\sum_{j=1}^n d_{i,j} X_{i,j}\right)_{i \in [\![1,n]\!]}$.
On s'intéresse à la somme où intervient $d_{i,j}$: $\sum_{k=1}^n d_{i,k} X_k$.
Alors $\mathbb{P}(DX = 0) \leq \mathbb{P}(\sum_{k=1}^n d_{i,k} X_k) = 0$, on définit y tel que $\sum_{k=1}^n d_{i,k} X_k = y + d_{i,j} X_j$.
Si $y = 0$, alors $y + d_{i,j} X_j = 0 \iff X_j = 0$. Sinon, $y + d_{i,j} X_j = 0 \iff X_j = 1$.

$$\begin{aligned}\mathbb{P}(\sum_{k=1}^n d_{i,k} X_k = 0) &= \mathbb{P}(y + d_{i,j} X_j = 0) = \mathbb{P}(y = 0)\mathbb{P}(y + d_{i,j} X_j = 0 \mid y = 0) + \mathbb{P}(y \neq 0)\mathbb{P}(y + d_{i,j} X_j = 0 \mid y \neq 0) \\ &= \mathbb{P}(y = 0)\mathbb{P}(X_j = 0) + \mathbb{P}(y \neq 0)\mathbb{P}(X_j = 1) = \frac{1}{2}\mathbb{P}(y = 0) + \frac{1}{2}\mathbb{P}(y \neq 0) = \frac{1}{2}.\end{aligned}$$

Si la réponse attendue est oui, l'algorithme est correct, sinon, il a moins d'une chance sur deux de faire erreur.
Si on répète l'algorithme k fois et qu'on renvoie faux ssi il a répondu faux à chaque appel, la probabilité d'erreur est inférieure à $\frac{1}{2^k}$. C'est toujours un algorithme en $O(n^2)$.

Définition 2

Cet algorithme est de type **Monte-Carlo**, c'est une classe d'algorithmes qui possèdent une probabilité bornée de renvoyer une mauvaise solution.

Définition 3

Il existe une autre classe d'algorithmes probabilistes : **Las Vegas**. Ceux-ci renvoient toujours le résultat attendu mais leur temps d'éxecution suit une variable aléatoire.

Exemple 4

Le tri rapide avec choix aléatoire du pivot est un algorithme de Las Vegas.

Solution :

On analyse la complexité moyenne du tri rapide.
On sait que dans le pire des cas, il s'agit de $O(n^2)$, et dans le cas moyen de $O(n \log n)$.
On considère un tableau $\mathscr{T}$ à trier et $\mathscr{T}'$ sa version triée. On mesure X le nombre de comparaisons.
La complexité du tri rapide est proportionnelle à son nombre de comparaisons.
On note $X_{i,j}$ le nombre de fois qu'ont été comparés le $i^{\text{ème}}$ élément et $j^{\text{ème}}$ élément de $\mathscr{T}'$.
Lorsqu'une comparaison est réalisée, elle implique forcément le pivot. Donc $X_{i,j} \in \{0, 1\}$.
Le cas 0 est obtenu lorsqu'ils ont été séparés dans deux tableaux différents.

$$X = \sum_{1 \leq i,j \leq n} X_{i,j} \quad \text{et} \quad \mathbb{E}(X) = \sum_{1 \leq i,j \leq n} \mathbb{E}(X_{i,j}) = \sum_{1 \leq i,j \leq n} \mathbb{P}(X_{i,j} = 1)$$

$X_{i,j}$ vaut 1 lorsque $\mathscr{T}'[i]$ ou $\mathscr{T}'[j]$ est le premier pivot choisi parmi les $\mathscr{T}'[k]$ où $k \in [\![i, j]\!]$.
Puisque ce choix est fait uniformément, $\mathbb{P}(X_{i,j} = 1) = \frac{2}{|[\![i,j]\!]|} = \frac{2}{j-i+1}$.

$$\mathbb{E}(X) = \sum_{i=1}^n \sum_{j=i+1}^n \frac{2}{j - i + 1} = \sum_{i=1}^n \sum_{k=1}^{n-i} \frac{2}{k+1} \leq 2 \sum_{i=1}^n H_n \leq 2n \log n.$$

Proposition 5: Lien entre Las Vegas et Monte-Carlo

Soit P un problème que l'on sait résoudre à l'aide d'un algorithme Las Vegas. Cet algorithme a une complexité moyenne en $f(n)$ où n est la taille de l'instance. On peut fixer T un temps de fin de calcul, on lance l'algorithme pendant un temps T et s'il n'a pas fini de s'exécuter, on renvoie n'importe quelle valeur.

$$\mathbb{P}(\text{bonne réponse}) \geq \mathbb{P}(\text{fin avant } T)$$

Si on note X la variable aléatoire que suit le temps d'exécution de l'algorithme de Las Vegas.
On veut être sûr que l'évènement $X \geq T$ se produise, T doit donc dépendre de $f(n)$. En posant $T = \alpha f(n)$:

$$\mathbb{P}(X \geq T) \leq \frac{\mathbb{E}(X)}{\alpha f(n)} = \frac{1}{\alpha} \quad \text{(Markov)}$$

Définition 6: Calcul de la médiane.

On cherche à calculer la médiane d'un tableau $\mathscr{T}$ à n éléments. On définit $(Y_i)_{i \leq n}$ une famille de variables de Bernoulli de paramètre $n^{-1/4}$. On restreint $\mathscr{T}$ aux éléments tels que $Y_i = 1$, on note $\mathscr{T}'$ cette restriction, si $|\mathscr{T}'| \geq \frac{2}{3}n^{3/4}$, on renvoie Erreur 1. Sinon on trie $\mathscr{T}'$.
On choisit alors les éléments $\lfloor \frac{1}{2}n^{3/4} - \sqrt{n} \rfloor$ et $\lfloor \frac{1}{2}n^{3/4} + \sqrt{n} \rfloor$, qu'on note d et u.
Si le rang de d dans $\mathscr{T}$ est supérieur à $\frac{n}{2}$, ou celui de u est inférieur à $\frac{n}{2}$, on renvoie Erreur 2.
On trie les éléments de $\mathscr{T}$ compris entre d et u ($\mathscr{T}''$), s'il y en a plus de $\frac{4}{3}n^{3/4}$, on renvoie Erreur 3.
Sinon on renvoie l'élément $\lfloor \frac{n}{2} \rfloor - r_d$. Pour connaître le rang de d et u, il n'est pas nécessaire de trier, il suffit de parcourir $\mathscr{T}$ et de compter le nombre d'éléments inférieurs. Si l'algorithme ne renvoie pas d'erreur, il renvoie la médiane en temps linéaire.
On peut borner la probabilité de renvoyer une erreur par une constante $p < 1$, on construit l'algorithme Las Vegas relancant l'algorithme à chaque erreur. Le temps d'exécution est proportionnel au nombre d'essais nécessaires pour ne pas avoir d'erreurs, le nombre d'essais suit donc une loi géométrique.
Son espérance est $\frac{1}{p}$, en moyenne, on exécute l'algorithme $\frac{1}{p}$ fois, la complexité en moyenne est en $O(\frac{n}{p}) = O(n)$.

Proposition 7

Pour transformer un algorithme de Monte-Carlo en Las Vegas, on le répète jusqu'à avoir la bonne réponse. Cette construction n'a de sens que si le test qui détermine si une répponse est bonne se fait rapidement. Pour la vérification du produit de matrices, le test serait en $O(n^3)$, rendant l'algorithme probabilisite inutile.

Définition 8: Lien entre algorithmes d'approximations et algorithmes probabilistes.

Instance. Une formule sous FNC dont les clauses sont de taille 3, et de variables distinctes.
Solution. Une valuation.
Fonction à maximiser : nombre de clauses valuées à vrai.
Étudions le nombre de clauses satisfaites par une valuation aléatoire.

Preuve :

On note $C_i = 1$ si la $i^{\text{ème}}$ clause est satisfaite. On s'intéresse à $\sum C_i$.
On a $\mathbb{P}(C_i = 1) = 1 - \mathbb{P}(C_i = 0) = 1 - \frac{1}{8} = \frac{7}{8}$, donc $\mathbb{E}(\sum C_i) = \frac{7n}{8}$.
Il existe donc une valuation satisfaisant $\frac{7}{8}$ème des clauses, lorsque $n < 8$, la formule est satisfiable.
Ici, $f(s_{\text{OPT}}) \leq n$, on peut répéter l'algorithme jusqu'à obtenir une valuation qui satisfait $\frac{7n}{8}$ des clauses.
Alors $\frac{7}{8}f(s_{\text{OPT}}) \leq f(s_A) \leq f(s_{\text{OPT}})$. Si on demande à satisfaire $\frac{3}{4}$ clauses, on a une probabilité de succès supérieure à $\frac{1}{2}$. En effet, si X compte le nombre de clauses non-satisfaites, alors $\mathbb{E}(X) = \frac{n}{8}$.
Donc $\mathbb{P}(\sum C_i \geq \frac{6n}{8}) = P(X < \frac{2n}{8}) \geq \frac{1}{2}$, il faudra donc en moyenne 2 essais pour trouver une valuation qui satisfait $\frac{6n}{8}$ clauses.